Software Dependency Standard - (SDS)

Software Dependency Standard - (SDS)

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Parent Standard

Governed Space: Software Dependency Governance

Standard ID: SLGA-SDS-003

Origin ID: OOF.PS.SLGA.03.01

Version: 1.0

Status: Canonical · Open Standard

Effective Date: 11.10.2026

Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

The Software Dependency Standard (SDS) governs the identification,

introduction, recognition, evaluation, authorization, maintenance, change, and continuity of software dependencies upon which a software system relies for its creation, operation, modification, or continued support.

SDS establishes the methodological conditions under which a software

dependency may be recognized as a governed relationship between a software system and an external or internal software component, service, library, platform, interface, execution environment, or other software-related resource.

It governs the relationship between a software dependency's identity,

origin, purpose, necessity, authority, compatibility, constraints, material risks, change conditions, evidence, and lifecycle continuity.

SDS distinguishes the technical ability to use a dependency from legitimate

authority to introduce it, the presence of a dependency from evidence that it is necessary or suitable, and the current functionality of a dependency from assurance that its availability, compatibility, or support will continue.

SDS applies across programming languages, development methodologies,

software architectures, deployment environments, AI-assisted development, autonomous software creation, and distributed software ecosystems.

Its principal output is:

Governed Software Dependency State

Its foundational distinction is:

Software Dependency Availability ≠ Governed Software Dependency

Acceptance.

Operational Role

The operational role of SDS is:

To ensure that software dependencies remain identifiable, justified, authorized, bounded, compatible, evidence-supported, and subject to reassessment as their technical, operational, organizational, or lifecycle conditions change.

SDS establishes the dependency-governance layer of SLGA™.

It governs the conditions under which a software system may rely on other software-related resources.

It does not prohibit the use of external libraries, open-source components, cloud services, automated development tools, or complex dependency networks.

Instead, it establishes the conditions under which reliance on those

resources can be recognized, evaluated, maintained, and reconsidered.

SDS recognizes that software dependency governance is not limited to the initial decision to include a component.

A dependency may remain technically functional while becoming unsupported, incompatible, unavailable, unmaintained, materially changed, or unsuitable for the software system's continued operation.

Core Question

How can software systems depend on internal and external software resources while preserving dependency identity, legitimate inclusion, justified reliance, compatibility, change control, evidence, and operational continuity throughout the software lifecycle?

Position in Architecture Governance Flow

The SLGA™ governance flow is:

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting:

Software Lifecycle Control

SDS governs:

Software Dependencies

It is the third Parent Standard of SLGA™.

Its principal upstream governance references include:

Governed Software Requirements Baseline

from Software Requirements Standard (SRS).

Governed Software Creation State

from Software Creation Standard (SCS).

Its principal output is:

Governed Software Dependency State

That state supports:

Software Configuration Standard (SCFS)

Software Quality Standard (SQS)

Software Security Standard (SSS)

Software Release Standard (SRLS)

Software Deployment Standard (SDPS)

Software Change Management Standard (SCMS)

Software Retirement Standard (SRTS)

Software Lifecycle Control Standard (SLCS)

Dependency governance may begin during requirements formation or software creation and continue through deployment, maintenance, and retirement.

The SLGA™ governance flow represents relationships between governed

lifecycle domains rather than a mandatory waterfall sequence.

• Governed Space

Software Dependency Governance

This space governs the conditions under which a software system

establishes, uses, changes, and terminates material dependency relationships.

Its principal governance dimensions include:

• Dependency identification

• Dependency purpose

• Dependency necessity

• Dependency origin

• Dependency ownership and control references

• Dependency introduction authority

• Dependency inclusion conditions

• Dependency compatibility

• Dependency relationship structure

• Dependency criticality

• Dependency risk

• Dependency constraints

• Dependency change

• Dependency continuity

• Dependency evidence

• Dependency governance states

• Dependency reassessment

• Dependency lifecycle handoff

The governed space is independent of any particular package manager, software vendor, repository, programming language, or development platform.

Software Dependency Governance Object

Every SDS assessment shall identify the Software Dependency Governance

Object.

The object represents a dependency relationship or defined dependency set

subject to governance.

It should identify:

Dependency Governance Identity: The identifiable dependency assessment.

Dependent Software Object: The software system relying on the dependency.

Dependency Identity: The software resource upon which reliance exists.

Dependency Type: The relevant dependency classification.

Dependency Purpose: The reason the dependency is used.

Dependency Origin: The source or provenance reference.

Dependency Authority: The applicable introduction or use authority.

Dependency Scope: The permitted dependency relationship.

Dependency Version or State: The relevant technical dependency condition.

Dependency Compatibility: The applicable compatibility relationships.

Dependency Criticality: The significance of the dependency to the

software system.

Dependency Risk: Material dependency-related risks.

Dependency Evidence: Evidence supporting the dependency relationship.

Dependency Governance State: The current recognized dependency condition.

The dependency governance object shall remain distinguishable from the

software component itself.

Dependency Artifact Identity ≠ Dependency Relationship Identity.

Software Dependency Recognition

SDS shall establish the conditions under which a software-related resource

is recognized as a dependency.

A dependency exists where a software system materially relies on another resource for a defined creation, execution, maintenance, or lifecycle function.

Recognition may concern:

• Direct dependencies

• Transitive dependencies

• Runtime dependencies

• Development dependencies

• Build dependencies

• Service dependencies

• Platform dependencies

• Interface dependencies

• Operational support dependencies

A resource shall not be excluded from governance solely because it is not listed in a conventional software package manifest.

Package Manifest Entry ≠ Complete Dependency Recognition.

Dependency Identity Governance

SDS shall establish identifiable references for material software

dependencies.

Relevant identity information may include:

• Canonical dependency name

• Supplier or source reference

• Package or service identity

• Version

• Release identity

• Repository reference

• Relevant artifact identifiers

• Applicable origin evidence

• Material identity uncertainty

Dependency identity shall remain sufficiently precise to distinguish

materially different versions or resources.

Dependency Name ≠ Verified Dependency Identity.

Dependency Origin Governance

SDS shall preserve relevant origin and provenance references for material

dependencies.

Origin may concern:

• Software publisher

• Maintainer

• Repository

• Distribution source

• Package registry

• Internal development organization

• External supplier

• Generated component source

• Relevant transformation history

Origin evidence shall be evaluated according to the claim it supports.

The existence of a publisher name does not independently establish authenticity, legitimacy, or continued maintenance.

Claimed Dependency Origin ≠ Verified Dependency Provenance.

Detailed origin and artifact integrity governance remains within competent OOF® identity and SCS artifact integrity domains.

Dependency Purpose Governance

SDS shall establish the purpose for which a dependency is introduced or maintained.

Relevant purposes may include:

• Implementing required functionality

• Supporting execution

• Providing an interface

• Enabling development

• Supporting testing

• Providing build capabilities

• Enabling interoperability

• Supporting operational continuity

A dependency may provide useful functionality without being necessary or suitable for the governed software objective.

Dependency Utility ≠ Dependency Necessity.

Dependency Necessity Governance

SDS shall evaluate whether material reliance on a dependency is justified relative to the software system's requirements and intended operation.

Relevant considerations may include:

• Functional necessity

• Alternative implementation possibilities

• Technical constraints

• Operational conditions

• Maintenance implications

• Resource requirements

• Integration complexity

• Dependency criticality

SDS does not require every dependency to be indispensable.

It requires material dependency decisions to remain sufficiently justified

for their intended governance context.

Dependency Convenience ≠ Governed Dependency Justification.

Dependency Introduction Authority

SDS shall identify the applicable authority basis for introducing a

dependency.

Relevant conditions may include:

• Authorized software creation objective

• Permitted dependency categories

• Approved sources

• Applicable organizational restrictions

• Delegated development authority

• Required approval conditions

• Relevant licensing or contractual references

A developer or autonomous agent may possess technical permission to install a dependency without possessing legitimate authority to introduce it into the governed software system.

Package Installation Capability ≠ Dependency Introduction Authority.

Dependency Inclusion Conditions

SDS shall establish the conditions under which a dependency may be included in a software system.

Relevant considerations may include:

• Dependency identity

• Applicable requirements

• Legitimate introduction authority

• Dependency purpose

• Compatibility

• Material risks

• Applicable constraints

• Evidence sufficiency

• Required reassessment conditions

A dependency may be conditionally accepted.

Acceptance may depend on restrictions, replacement obligations, monitoring, or additional assessment.

Dependency Inclusion ≠ Unconditional Dependency Acceptance.

Direct Dependency Governance

SDS shall identify dependencies directly introduced or referenced by the

governed software system.

Direct dependencies may include:

• Imported libraries

• Declared packages

• Linked software components

• Explicit service integrations

• Required runtime platforms

• Direct development tool dependencies

Direct dependency recognition shall preserve the relationship between the

dependent software object and the relied-upon resource.

Declared Direct Dependency ≠ Fully Governed Dependency.

Transitive Dependency Governance

SDS shall identify material dependencies introduced indirectly through other dependencies.

A relevant chain may be:

Software System → Direct Dependency → Transitive Dependency → Further

Dependency

Transitive dependencies may create material risk or continuity conditions even where the software developer did not explicitly select them.

SDS shall preserve relevant relationship depth and uncertainty.

Indirect Dependency ≠ Irrelevant Dependency.

Dependency Network Governance

Software systems may rely on complex dependency networks.

SDS shall support representation of material dependency relationships as a

structured network.

Relevant relationships may include:

• Direct reliance

• Indirect reliance

• Shared dependency

• Conditional dependency

• Alternative dependency

• Critical dependency

• Version-specific dependency

• Service-to-service dependency

Network representation shall reflect supported dependency relationships

rather than inferred associations alone.

Dependency Graph Existence ≠ Verified Dependency Network.

Dependency Scope Governance

SDS shall identify the scope within which a dependency is applicable.

Scope may concern:

• Software component

• Application

• Development environment

• Build environment

• Runtime environment

• Deployment environment

• Geographic or organizational context

• Specific feature

• Defined software version

A dependency accepted for one scope shall not automatically be considered

accepted for every other scope.

Dependency Acceptance in One Context ≠ Universal Dependency Acceptance.

Dependency Version Governance

SDS shall identify the relevant versions or states of material dependencies.

Version governance may consider:

• Version identity

• Release status

• Compatibility

• Maintenance status

• Support conditions

• Relevant changes

• Version constraints

• Evidence of actual use

A dependency name without a sufficiently identified version may be inadequate for material governance assessment.

Dependency Version Label ≠ Verified Dependency State.

Dependency Compatibility Governance

SDS shall evaluate material compatibility relationships between a

dependency and the software system relying on it.

Compatibility may concern:

• Programming interfaces

• Data formats

• Runtime environments

• Operating systems

• Software versions

• Build conditions

• Communication protocols

• Relevant behavioral assumptions

A dependency may be technically installable while remaining incompatible with the intended software system.

Installability ≠ Dependency Compatibility.

Dependency Constraint Governance

SDS shall identify material constraints affecting dependency use.

Constraints may include:

• Version restrictions

• Platform requirements

• Licensing conditions

• Resource requirements

• Interface limitations

• Organizational restrictions

• Geographic availability

• Supplier conditions

• Operational dependencies

Constraint recognition shall preserve the distinction between technical

restrictions and legitimate governance boundaries.

Technical Constraint ≠ Complete Governance Constraint.

Dependency Criticality Governance

SDS shall assess the significance of material dependencies to the software

system.

Criticality may depend on:

• Functional importance

• Failure consequences

• Availability of alternatives

• Replacement difficulty

• Operational reliance

• Dependency concentration

• Recovery conditions

• Downstream impact

A small software library may be operationally critical.

A large dependency may be comparatively replaceable.

Dependency Size ≠ Dependency Criticality.

Dependency Risk Governance

SDS shall identify material risks arising from dependency relationships.

Relevant risk categories may include:

• Availability risk

• Compatibility risk

• Maintenance risk

• Supplier risk

• Concentration risk

• Replacement risk

• Version-change risk

• Operational continuity risk

• Integrity uncertainty

• Relevant security risk references

SDS governs the dependency relationship and its risk implications.

Detailed software security assessment remains within Software Security Standard (SSS).

Dependency Risk Identification ≠ Dependency Risk Resolution.

Dependency Concentration Governance

SDS shall identify material reliance on common dependencies or dependency

providers.

Concentration may occur where:

• Multiple applications rely on one library

• Several services depend on one platform

• Numerous components rely on one external API

• A critical software ecosystem depends on one maintainer

• Multiple dependency paths converge on one shared resource

Concentration may create correlated failure or continuity exposure.

Multiple Dependency Paths ≠ Independent Dependency Resilience.

Dependency Substitutability

SDS shall assess whether a material dependency can be replaced without unacceptable disruption to the governed software objective.

Relevant considerations may include:

• Available alternatives

• Interface compatibility

• Data portability

• Migration effort

• Replacement authority

• Technical adaptation

• Operational disruption

• Evidence of replacement feasibility

The existence of an alternative product does not establish practical substitutability.

Alternative Availability ≠ Effective Dependency Substitutability.

Dependency Continuity Governance

SDS shall preserve the conditions under which material dependency

relationships may remain supportable over time.

Continuity may depend on:

• Supplier availability

• Maintainer activity

• Version support

• Technical compatibility

• Access conditions

• Operational availability

• Replacement readiness

• Dependency change monitoring

Current functionality does not establish future continuity.

Current Dependency Functionality ≠ Dependency Continuity Assurance.

Dependency Maintenance Governance

SDS shall identify material maintenance conditions affecting software

dependencies.

Relevant considerations may include:

• Maintainer identity

• Maintenance activity

• Support status

• Update availability

• Known compatibility changes

• Deprecation

• End-of-support conditions

• Relevant maintenance evidence

A dependency may remain functional after active maintenance has ended.

Functional Dependency ≠ Maintained Dependency.

Dependency Update Governance

SDS shall establish the conditions under which material dependency updates

are evaluated and recognized.

An update may affect:

• Dependency identity

• Version

• Compatibility

• Functionality

• Security exposure

• Integration requirements

• Operational continuity

• Applicable constraints

An available update shall not automatically be treated as appropriate or

authorized.

Update Availability ≠ Update Authorization.

Dependency Change Governance

SDS shall preserve material changes in dependency relationships.

Changes may include:

• Addition

• Removal

• Replacement

• Version change

• Source change

• Supplier change

• Interface change

• Scope change

• Dependency role change

• Continuity-condition change

Material dependency changes may require reassessment of downstream software lifecycle states.

Dependency Change ≠ Isolated Technical Modification.

Detailed lifecycle change control remains within Software Change Management Standard (SCMS).

Dependency Replacement Governance

SDS shall establish the conditions under which a material dependency may be replaced.

Relevant considerations may include:

• Replacement objective

• Existing dependency identity

• Candidate replacement identity

• Compatibility

• Migration conditions

• Authority

• Relevant risks

• Continuity

• Evidence

Replacement shall not be recognized solely because the new dependency

provides similar functionality.

Functional Similarity ≠ Governed Dependency Equivalence.

Dependency Removal Governance

SDS shall identify the conditions under which a dependency may be removed.

Relevant considerations may include:

• Continued reliance

• Transitive dependency effects

• Compatibility implications

• Software functionality

• Operational conditions

• Relevant evidence

• Downstream lifecycle impact

Removing a declared dependency does not automatically establish that the software system no longer relies on it.

Manifest Removal ≠ Dependency Elimination.

Dependency Obsolescence

SDS shall identify material dependency obsolescence.

Obsolescence may arise through:

• End of maintenance

• End of support

• Platform incompatibility

• Supplier discontinuation

• Technological replacement

• Unavailable updates

• Unsupported execution environments

An obsolete dependency may remain operational.

Its continued use requires assessment of the applicable dependency

governance conditions.

Operational Dependency ≠ Sustainable Dependency.

Dependency Availability

SDS shall identify material availability conditions affecting dependency

reliance.

Availability may concern:

• Package registries

• Software repositories

• External services

• License servers

• Runtime platforms

• Build services

• Required distribution channels

Availability at one point in time does not establish continued access.

Dependency Availability Today ≠ Dependency Availability Tomorrow.

Dependency Supplier and Maintainer Governance

SDS shall preserve relevant relationships with dependency suppliers,

publishers, and maintainers.

Relevant conditions may include:

• Supplier identity

• Maintainer identity

• Distribution responsibility

• Support arrangements

• Relevant control changes

• Maintenance continuity

• Material supplier uncertainty

SDS does not independently establish contractual liability or ownership rights.

Supplier Identification ≠ Supplier Accountability Determination.

Autonomous Dependency Introduction

AI agents and autonomous development systems may introduce dependencies during software creation.

SDS shall preserve the relevant dependency-governance conditions regardless

of the actor performing introduction.

Relevant considerations may include:

• Authorized development objective

• Dependency selection

• Introduction authority

• Source

• Version

• Compatibility

• Material risk

• Resulting software reliance

• Evidence

An agent's ability to install packages does not establish unlimited authority to expand the dependency network.

Autonomous Dependency Selection ≠ Autonomous Dependency Authorization.

Detailed autonomous execution control remains within ASCCM.

AI-Generated Dependency Relationships

AI-assisted software creation may introduce dependency relationships that are not explicitly recognized by the developer.

Examples may include:

• Generated imports

• Suggested package installations

• External service calls

• Generated integration code

• Model-selected frameworks

• Automated dependency resolution

SDS shall identify material dependencies arising from generated software.

The fact that a dependency was introduced through AI-generated code does not eliminate the need for governance.

AI-Generated Dependency ≠ Automatically Governed Dependency.

Dependency Evidence

SDS shall preserve evidence supporting material dependency relationships.

Relevant evidence may include:

• Dependency manifests

• Package lockfiles

• Artifact identifiers

• Source references

• Version records

• Integration records

• Compatibility assessments

• Dependency graphs

• Supplier information

• Maintenance records

• Change records

• Relevant approval references

Evidence requirements shall reflect dependency materiality.

Dependency Record Existence ≠ Dependency Evidence Sufficiency.

Dependency Evidence Integrity

SDS shall evaluate the integrity of material dependency evidence.

Relevant considerations may include:

• Evidence origin

• Content integrity

• Temporal relevance

• Version correspondence

• Source reliability

• Preservation history

• Material contradictions

• Evidence completeness

A generated dependency manifest may not reflect every runtime or service dependency.

Generated Dependency Manifest ≠ Verified Dependency Reality.

Dependency Traceability

SDS shall preserve material relationships between dependencies and the

software systems that rely on them.

Relevant traceability may include:

Software Object → Dependency → Version → Purpose → Authority →

Compatibility → Risk → Evidence

Dependency traceability shall support reconstruction of material reliance

relationships.

A recorded dependency link does not independently establish compatibility or acceptance.

Recorded Dependency Link ≠ Governed Dependency Traceability.

Dependency Uncertainty

Dependency governance may remain uncertain where:

• Dependency identity is incomplete

• Transitive relationships are unknown

• Origin evidence is missing

• Version information is unreliable

• Compatibility is unresolved

• Supplier continuity is uncertain

• Material constraints are unclear

• Actual runtime reliance cannot be reconstructed

SDS shall preserve material uncertainty.

Uncertain Dependency ≠ Verified Dependency.

Uncertainty in one relationship shall not automatically invalidate every

other supported dependency relationship.

Dependency Conflict

Dependency conflicts may arise where:

• Required versions are incompatible

• Multiple components require contradictory interfaces

• Dependency sources disagree

• Supplier conditions conflict

• Integration constraints cannot be reconciled

• Recorded dependencies differ from actual runtime conditions

SDS shall identify material conflicts and preserve supporting evidence.

Technical resolution of a conflict does not automatically establish that the resulting dependency change was authorized.

Technical Dependency Resolution ≠ Governance Conflict Resolution.

Dependency Gap

A Dependency Governance Gap exists where a material dependency condition cannot be sufficiently established.

Examples include:

• Unknown dependency origin

• Missing version identity

• Unrecognized transitive dependency

• Unresolved introduction authority

• Unsupported compatibility assumption

• Missing maintenance information

• Insufficient continuity evidence

SDS shall identify the gap and its relevance to the governed software

system.

Dependency Governance Gap ≠ Automatic Software Invalidity.

Critical Dependency Gap

A Critical Dependency Governance Gap exists where missing or contradictory information prevents reliable assessment of a materially consequential dependency relationship.

Examples may include:

• An unidentified component supporting a critical system function

• A major external service with unknown continuity conditions

• An unverified dependency introduced into safety-relevant software

• A critical package with unresolved origin and version evidence

• A transitive dependency whose failure could materially affect several

systems.

SDS shall preserve the critical gap and support competent escalation.

It does not independently determine software release rejection, operational suspension, or liability.

Critical Dependency Gap Identification ≠ Operational Decision Authority.

Dependency Drift

Dependency Drift occurs when actual software dependency relationships materially diverge from the recognized dependency state.

Possible causes include:

• Automatic package updates

• Unrecorded installations

• Transitive dependency changes

• External service changes

• Build environment changes

• Supplier changes

• Runtime dependency substitutions

• AI-generated modifications

SDS shall identify material drift and support reassessment.

Previously Governed Dependency ≠ Permanently Governed Dependency.

Dependency Governance States

SDS shall support identifiable dependency governance states.

Possible states include:

Identified: The dependency relationship has been recognized.

Under Assessment: Material dependency conditions are being evaluated.

Conditionally Accepted: Acceptance depends on explicit conditions.

Accepted: Applicable dependency governance conditions are sufficiently established.

Restricted: Dependency use is subject to defined limitations.

Evidence-Limited: Available evidence is insufficient.

Origin-Unresolved: Material origin conditions remain unsupported.

Compatibility-Unresolved: Material compatibility conditions remain

uncertain.

Authority-Unresolved: Applicable introduction or use authority cannot be

sufficiently established.

Continuity-at-Risk: Material continuity conditions are threatened.

Conflicted: Material dependency conditions contradict one another.

Rejected: The dependency has not satisfied applicable acceptance

conditions.

Superseded: Another governed dependency state has replaced the previous

state.

Retired: The dependency relationship has been formally ended within the applicable scope.

Unresolved: Available information does not support a sufficiently determined state.

These states shall not automatically be interpreted as software security,

quality, or release determinations.

Dependency Accepted ≠ Software Product Accepted.

Dependency Acceptance

SDS shall determine whether a dependency may be recognized as accepted for its intended governance scope.

Acceptance should consider:

• Dependency identity

• Origin

• Purpose

• Necessity

• Introduction authority

• Inclusion conditions

• Compatibility

• Constraints

• Criticality

• Material risks

• Continuity

• Evidence sufficiency

• Uncertainty

• Required reassessment conditions

Acceptance may be conditional or restricted.

Acceptance shall remain specific to the dependency relationship and context

assessed.

Dependency Acceptance ≠ Universal Dependency Approval.

Dependency Reassessment

SDS shall support reassessment when material dependency conditions change.

Triggers may include:

• New dependency versions

• Supplier changes

• Maintenance changes

• Support termination

• Compatibility changes

• New vulnerabilities

• Material operational incidents

• Dependency replacement

• Dependency concentration changes

• New evidence

• Dependency drift

• Changed software requirements

Reassessment shall preserve historical dependency states where necessary for reconstruction.

Dependency Continuity Across Software Lifecycle

SDS shall preserve material dependency relationships throughout the

relevant software lifecycle.

Continuity may involve:

• Software creation

• Configuration

• Quality assessment

• Security assessment

• Release

• Deployment

• Maintenance

• Change

• Retirement

Dependency governance shall not be treated as complete merely because the

initial software creation activity has ended.

Dependency Acceptance at Creation ≠ Dependency Acceptance for Every Later

Lifecycle State.

Dependency Lifecycle Handoff

SDS shall provide relevant dependency-governance information to competent

downstream SLGA™ processes.

Relevant information may include:

• Dependency identity

• Dependency version

• Dependency relationships

• Acceptance conditions

• Compatibility

• Material constraints

• Criticality

• Risks

• Continuity conditions

• Evidence

• Uncertainty

• Reassessment triggers

SDS does not independently perform downstream quality, security, release, deployment, or retirement assessments.

Dependency Governance Completion ≠ Downstream Lifecycle Governance

Completion.

Core Modules

SDS contains five Core Modules. CIM — Component Identification Module

Governs the identification and classification of material software

dependency components, including their identities, versions, roles, origins, and relationships to dependent software objects.

Its principal focus is:

Governed Software Component Identification

DOTM — Dependency Origin & Traceability Module

Governs the origin, provenance, source relationships, and

evidence-supported traceability of software dependencies across direct, transitive, generated, and externally supplied dependency paths.

Its principal focus is:

Governed Dependency Origin & Traceability

DSM — Dependency Suitability Module

Governs the assessment of dependency necessity, appropriateness,

compatibility, constraints, and suitability for the intended software context.

Its principal focus is:

Governed Dependency Suitability

CICM — Component Introduction & Change Module

Governs the introduction, inclusion, modification, replacement, update, and

removal of software dependencies under applicable authority, scope, and change conditions.

Its principal focus is:

Governed Component Introduction & Change

DCRM — Dependency Continuity & Risk Module

Governs material dependency risks, criticality, maintenance conditions,

concentration, substitutability, availability, and continuity across the software lifecycle.

Its principal focus is:

Governed Dependency Continuity & Risk

These modules establish the internal SDS governance flow:

Component Identification → Dependency Origin & Traceability → Dependency

Suitability → Component Introduction & Change → Dependency Continuity & Risk

The flow represents the principal governance relationships between the

modules.

It does not require every dependency assessment to follow a fixed chronological sequence.

Software Dependency Governance Principles

SDS establishes the following principles. Principle 1 — Dependency Identity Before Acceptance

A dependency must be sufficiently identified before its governance state can be reliably assessed. Principle 2 — Reliance Must Be Recognized

Material dependency relationships must not remain invisible merely because they are indirect, generated, or outside conventional package manifests. Principle 3 — Introduction Requires Legitimate Authority

Technical installation capability does not establish permission to introduce a dependency. Principle 4 — Purpose Must Remain Connected to Requirements

Material dependencies must remain connected to a justified software purpose or applicable requirement. Principle 5 — Suitability Is Context-Dependent

A dependency suitable for one software context may be unsuitable for another. Principle 6 — Compatibility Is Not Acceptance

Technical compatibility does not independently establish legitimate dependency use. Principle 7 — Dependency Networks Matter

Direct and transitive relationships may create material risks that are not visible when components are assessed individually. Principle 8 — Continuity Must Be Reassessed

A dependency accepted today may become unsuitable, unsupported, or unavailable later. Principle 9 — Evidence Before Governance Claims

Dependency acceptance must remain supported by evidence appropriate to the claim. Principle 10 — Dependency Acceptance Is Not Product Certification

A governed dependency state does not establish the correctness, security, or release readiness of the dependent software system.

Governance Requirements

SDS establishes the following Parent Standard governance requirements.

SDS-GR-01 — Dependency Object Identification

Material dependency relationships shall be identifiable.

SDS-GR-02 — Dependency Identity

Material dependencies shall possess sufficiently distinguishable identities

and relevant version references.

SDS-GR-03 — Dependency Origin

Relevant dependency origin and provenance conditions shall be identifiable

or explicitly unresolved.

SDS-GR-04 — Dependency Purpose

Material dependencies shall remain connected to an identifiable software

purpose.

SDS-GR-05 — Dependency Necessity

Material reliance shall be assessed against applicable requirements and

software objectives.

SDS-GR-06 — Introduction Authority

Dependency introduction shall remain within applicable authority boundaries.

SDS-GR-07 — Dependency Scope

Dependency acceptance shall remain bounded by its applicable software

context.

SDS-GR-08 — Direct and Transitive Recognition

Material direct and transitive dependency relationships shall be

identifiable.

SDS-GR-09 — Compatibility

Material compatibility conditions shall be assessed and preserved.

SDS-GR-10 — Constraint Recognition

Relevant dependency restrictions and conditions shall be identified.

SDS-GR-11 — Criticality Assessment

Material dependency criticality shall be assessed.

SDS-GR-12 — Risk Recognition

Material dependency-related risks shall be identified and evaluated within the competent governance scope.

SDS-GR-13 — Dependency Change

Material dependency changes shall be recognized and subject to applicable

reassessment.

SDS-GR-14 — Continuity

Material dependency continuity conditions shall remain identifiable and

subject to reassessment.

SDS-GR-15 — Evidence Preservation

Dependency governance claims shall remain supported by sufficient evidence.

SDS-GR-16 — Governance State Recognition

Material dependency relationships shall have identifiable governance states.

SDS-GR-17 — Uncertainty and Conflict

Material dependency uncertainty, conflicts, and gaps shall remain explicit.

SDS-GR-18 — Lifecycle Handoff

Relevant dependency-governance information shall remain available to

competent downstream software lifecycle governance processes.

Governance Logic

The canonical SDS governance logic is:

Receive Governed Software Requirements Baseline

↓

Receive Governed Software Creation State

↓

Identify Software Dependency Governance Object

↓

Identify Dependent Software Object

↓

Discover Material Dependency Relationships

↓

Identify Direct and Transitive Dependencies

↓

Establish Dependency Identity and Version

↓

Assess Dependency Origin and Provenance References

↓

Identify Dependency Purpose

↓

Assess Dependency Necessity

↓

Identify Dependency Introduction Authority

↓

Establish Dependency Scope

↓

Identify Dependency Constraints

↓

Assess Dependency Compatibility

↓

Assess Dependency Criticality

↓

Identify Material Dependency Risks

↓

Assess Dependency Concentration and Substitutability

↓

Assess Maintenance and Availability Conditions

↓

Assess Dependency Continuity

↓

Preserve Dependency Evidence

↓

Identify Dependency Gaps, Conflicts, and Uncertainty

↓

Determine Dependency Acceptance Conditions

↓

Determine Dependency Governance State

↓

Establish Governed Software Dependency State

↓

Preserve Dependency Continuity and Traceability

↓

Handoff to Competent SLGA™ Lifecycle Governance Domains

↓

Reassess When Material Dependency Conditions Change

The principal output is:

Governed Software Dependency State

Governance Outputs

SDS may produce:

Software Dependency Governance Object

Dependent Software Object Reference

Dependency Identity Record

Dependency Classification

Dependency Origin Record

Dependency Purpose Record

Dependency Necessity Assessment

Dependency Introduction Authority Reference

Dependency Inclusion Conditions

Direct Dependency Record

Transitive Dependency Record

Dependency Network Map

Dependency Scope Record

Dependency Version Record

Dependency Compatibility Assessment

Dependency Constraint Record

Dependency Criticality Assessment

Dependency Risk Record

Dependency Concentration Assessment

Dependency Substitutability Assessment

Dependency Continuity Assessment

Dependency Maintenance Record

Dependency Update Assessment

Dependency Change Record

Dependency Replacement Record

Dependency Removal Record

Dependency Obsolescence Assessment

Dependency Availability Assessment

Dependency Supplier and Maintainer Record

Autonomous Dependency Introduction Record

AI-Generated Dependency Record

Dependency Evidence Record

Dependency Evidence Integrity Assessment

Dependency Traceability Record

Dependency Uncertainty State

Dependency Conflict Record

Dependency Governance Gap

Critical Dependency Governance Gap

Dependency Drift Record

Dependency Governance State

Dependency Acceptance Record

Dependency Reassessment Record

Dependency Lifecycle Continuity Record

Dependency Lifecycle Handoff Record

The principal output is:

Governed Software Dependency State

Critical Distinctions

SDS preserves the following distinctions:

Software Dependency Availability ≠ Governed Software Dependency Acceptance

Dependency Artifact Identity ≠ Dependency Relationship Identity

Package Manifest Entry ≠ Complete Dependency Recognition

Dependency Name ≠ Verified Dependency Identity

Claimed Dependency Origin ≠ Verified Dependency Provenance

Dependency Utility ≠ Dependency Necessity

Dependency Convenience ≠ Governed Dependency Justification

Package Installation Capability ≠ Dependency Introduction Authority

Dependency Inclusion ≠ Unconditional Dependency Acceptance

Declared Direct Dependency ≠ Fully Governed Dependency

Indirect Dependency ≠ Irrelevant Dependency

Dependency Graph Existence ≠ Verified Dependency Network

Dependency Acceptance in One Context ≠ Universal Dependency Acceptance

Dependency Version Label ≠ Verified Dependency State

Installability ≠ Dependency Compatibility

Technical Constraint ≠ Complete Governance Constraint

Dependency Size ≠ Dependency Criticality

Dependency Risk Identification ≠ Dependency Risk Resolution

Multiple Dependency Paths ≠ Independent Dependency Resilience

Alternative Availability ≠ Effective Dependency Substitutability

Current Dependency Functionality ≠ Dependency Continuity Assurance

Functional Dependency ≠ Maintained Dependency

Update Availability ≠ Update Authorization

Dependency Change ≠ Isolated Technical Modification

Functional Similarity ≠ Governed Dependency Equivalence

Manifest Removal ≠ Dependency Elimination

Operational Dependency ≠ Sustainable Dependency

Dependency Availability Today ≠ Dependency Availability Tomorrow

Supplier Identification ≠ Supplier Accountability Determination

Autonomous Dependency Selection ≠ Autonomous Dependency Authorization

AI-Generated Dependency ≠ Automatically Governed Dependency

Dependency Record Existence ≠ Dependency Evidence Sufficiency

Generated Dependency Manifest ≠ Verified Dependency Reality

Recorded Dependency Link ≠ Governed Dependency Traceability

Uncertain Dependency ≠ Verified Dependency

Technical Dependency Resolution ≠ Governance Conflict Resolution

Dependency Governance Gap ≠ Automatic Software Invalidity

Critical Dependency Gap Identification ≠ Operational Decision Authority

Previously Governed Dependency ≠ Permanently Governed Dependency

Dependency Accepted ≠ Software Product Accepted

Dependency Acceptance ≠ Universal Dependency Approval

Dependency Acceptance at Creation ≠ Dependency Acceptance for Every Later

Lifecycle State

Dependency Governance Completion ≠ Downstream Lifecycle Governance

Completion

Cross-Architecture Boundary

SDS governs software dependency relationships within SLGA™.

It does not independently establish general legal authority, software artifact integrity, software quality, software security, or release authorization.

Within SLGA™:

Software Requirements Standard (SRS) governs the requirements basis.

Software Creation Standard (SCS) governs the creation of software

artifacts.

Software Dependency Standard (SDS) governs the dependency relationships

upon which software relies.

Software Configuration Standard (SCFS) governs configuration states.

Software Quality Standard (SQS) governs software quality.

Software Security Standard (SSS) governs software security.

Software Release Standard (SRLS) governs release decisions.

Software Deployment Standard (SDPS) governs deployment conditions.

Software Change Management Standard (SCMS) governs broader lifecycle

changes.

Software Retirement Standard (SRTS) governs retirement.

Software Lifecycle Control Standard (SLCS) governs cross-cutting

lifecycle control.

Relevant interoperating OOF® architectures include:

GOA™ — Governance Architecture: General authority and delegation

governance.

AGA™ — Accountability Governance Architecture: Responsibility and

accountability relationships.

ASGA™ — Autonomous Systems Governance Architecture: Autonomous execution

governance.

AGI™ — Agent Governance Infrastructure: Agent permissions, delegation,

and tool-path governance.

OBIDENITY® — Origin-Bound Identity Governance Architecture: Applicable

origin, identity, ownership, and provenance relationships.

ORA™ — Operational Reality Architecture: Competent operational reality

assessment.

SDS preserves relevant dependency-governance relationships without

replacing the competent responsibilities of these architectures.

Canonical Boundary

SDS begins when a software system materially relies, or is proposed to rely, on an internal or external software-related resource.

SDS governs:

• Software dependency governance objects

• Dependency recognition

• Dependency identity

• Dependency origin

• Dependency purpose

• Dependency necessity

• Dependency introduction authority

• Dependency inclusion conditions

• Direct dependencies

• Transitive dependencies

• Dependency networks

- Dependency scope
- Dependency versions
- Compatibility
- Constraints
- Criticality
- Risks
- Concentration
- Substitutability
- Continuity
- Maintenance
- Updates
- Changes
- Replacement
- Removal
- Obsolescence
- Availability
- Supplier and maintainer relationships
- Autonomous dependency introduction
- AI-generated dependency relationships
- Dependency evidence
- Evidence integrity
- Dependency traceability
- Uncertainty
- Conflicts
- Governance gaps
- Critical governance gaps
- Drift
- Governance states
- Acceptance
- Reassessment
- Lifecycle continuity
- Downstream governance handoff

SDS does not independently determine:

- General legal or organizational authority
- Software requirements legitimacy
- Software creation integrity

• Complete artifact provenance

• Software configuration validity

• Software functional correctness

• Software security conformance

• Intellectual property ownership

• Contractual liability

• Release readiness

• Deployment authorization

• Retirement completion

SDS ends when the relevant dependency relationship or dependency set has been represented as a:

Governed Software Dependency State

with sufficient identity, purpose, authority, compatibility, risk,

evidence, continuity, uncertainty, and governance-state information for competent software lifecycle governance.

Canonical Closing Statement

Modern software rarely operates independently.

Applications rely on libraries, packages, platforms, interfaces, external

services, runtime environments, and complex networks of direct and transitive dependencies.

These dependencies may be introduced by human developers, automated tools, AI coding assistants, or autonomous software creation systems.

A dependency may be easy to install.

It may provide valuable functionality.

It may work correctly today.

Yet none of these conditions independently establishes that the dependency is legitimate, suitable, sufficiently understood, or sustainable.

A technically compatible library may have uncertain origin.

An available external service may become unavailable.

A maintained dependency may introduce material changes.

An autonomous coding agent may install a component without possessing the authority to introduce it.

And a software system may rely on dependencies that are not visible in its declared package manifest.

Software Dependency Availability ≠ Governed Software Dependency

Acceptance.

Package Installation Capability ≠ Dependency Introduction Authority.

Dependency Utility ≠ Dependency Necessity.

Installability ≠ Dependency Compatibility.

Current Dependency Functionality ≠ Dependency Continuity Assurance.

Dependency Accepted ≠ Software Product Accepted.

SDS establishes the methodological conditions under which software reliance

becomes an identifiable and governed relationship.

It ensures that dependency identity, origin, purpose, authority, suitability, constraints, risk, and continuity remain visible throughout the software lifecycle.

It does not require software to eliminate dependencies.

It requires material dependencies to remain governable.

SDS transforms software dependencies from technical implementation

conveniences into governed lifecycle relationships — ensuring that software systems may rely on external and internal resources without allowing technical availability, installation success, or present functionality to substitute for legitimate dependency acceptance, evidence-supported suitability, and sustainable operational continuity.

Module Architecture

→ CIM — Component Identification Module

→ DOTM — Dependency Origin & Traceability Module

→ DSM — Dependency Suitability Module

→ CICM — Component Introduction & Change Module

→ DCRM — Dependency Continuity & Risk Module

Related Documents

→ About Software Dependency Standard (SDS)